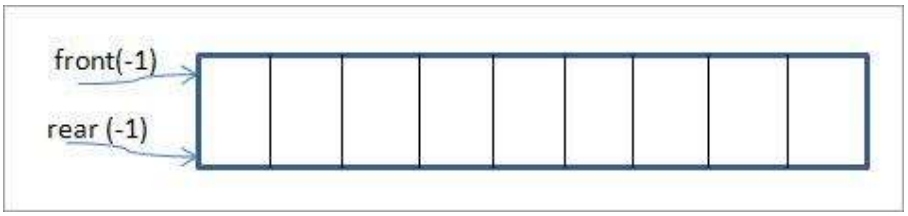
Lab# 05
Queue Data Structure In C++

5.1 Introduction To Queue In C++ With Illustration.

The queue is a basic data structure just like a stack. In contrast to stack that uses the LIFO approach, queue uses the FIFO (first in, first out) approach. With this approach, the first item that is added to the queue is the first item to be removed from the queue. Just like Stack, the queue is also a linear data structure.

In a real-world analogy, we can imagine a bus queue where the passengers wait for the bus in a queue or a line. The first passenger in the line enters the bus first as that passenger happens to be the one who had come first.

In software terms, the queue can be viewed as a set or collection of elements as shown below. The elements are arranged linearly.



We have two ends i.e. “front” and “rear” of the queue. When the queue is empty, then both the pointers are set to -1.

The “rear” end pointer is the place from where the elements are inserted in the queue. The operation of adding /inserting elements in the queue is called “enqueue”.

The “front” end pointer is the place from where the elements are removed from the queue. The operation to remove/delete elements from the queue is called “dequeue”.

When the rear pointer value is size-1, then we say that the queue is full. When the front is null, then the queue is empty.

5.2 Basic Operations

The queue data structure includes the following operations:

- **EnQueue:** Adds an item to the queue. Addition of an item to the queue is always done at the rear of the queue.
- **DeQueue:** Removes an item from the queue. An item is removed or de-queued always from the front of the queue.
- **isEmpty:** Checks if the queue is empty.
- **isFull:** Checks if the queue is full.
- **peek:** Gets an element at the front of the queue without removing it.

5.2.1 Enqueue

In this process, the following steps are performed:

- Check if the queue is full.
- If full, produce overflow error and exit.
- Else, increment 'rear'.
- Add an element to the location pointed by 'rear'.
- Return success.

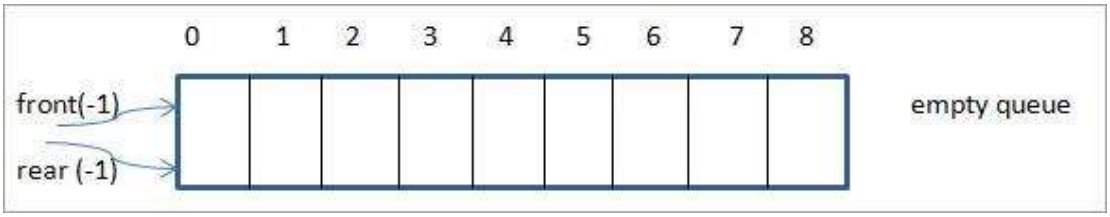
5.2.2 Dequeue

Dequeue operation consists of the following steps:

- Check if the queue is empty.
- If empty, display an underflow error and exit.
- Else, the access element is pointed out by 'front'.
- Increment the 'front' to point to the next accessible data.
- Return success.

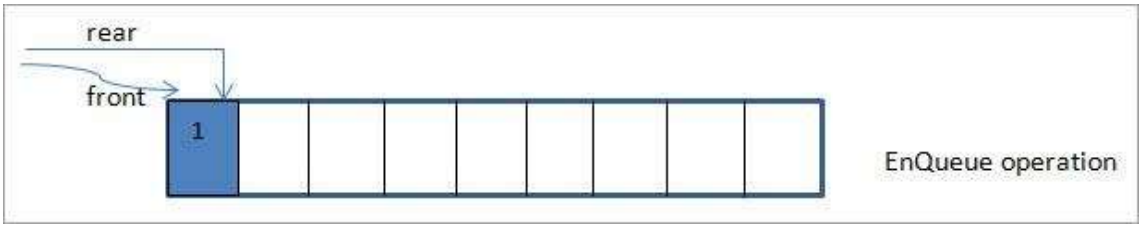
Next, we will see a detailed illustration of insertion and deletion operations in queue.

5.2.3 Illustration

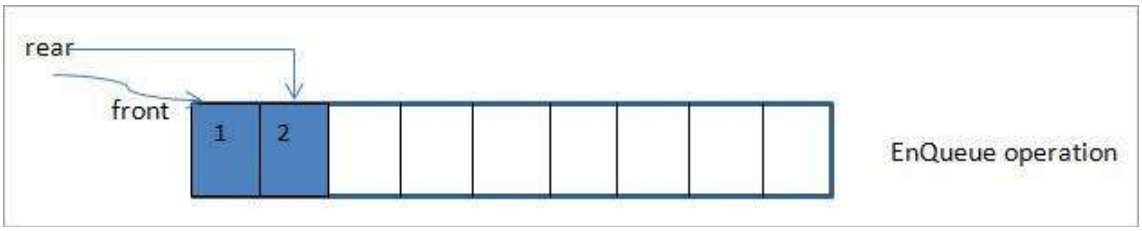


This is an empty queue and thus we have rear and empty set to -1.

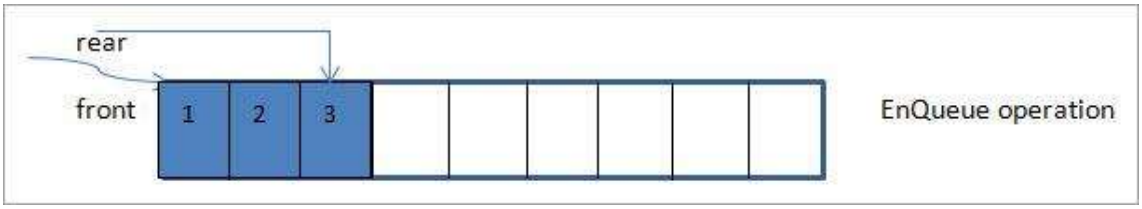
Next, we add 1 to the queue and as a result, the rear pointer moves ahead by one location.



In the next figure, we add element 2 to the queue by moving the rear pointer ahead by another increment.

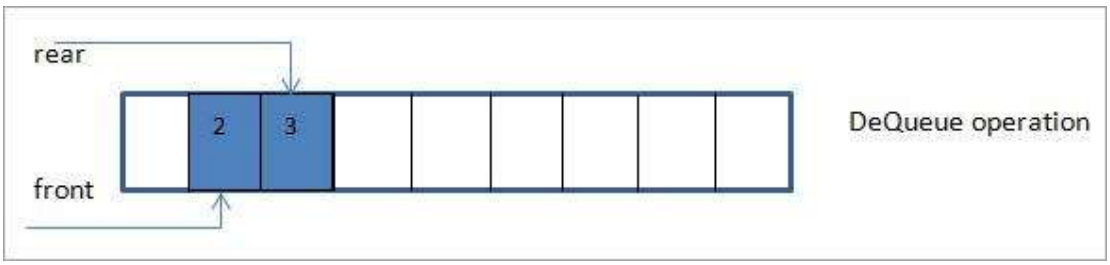


In the following figure, we add element 3 and move the rear pointer by 1.



At this point, the rear pointer has value 2 while the front pointer is at the 0th location.

Next, we delete the element pointed by the front pointer. As the front pointer is at 0, the element that is deleted is 1.



Thus the first element entered in the queue i.e. 1 happens to be the first element removed from the queue. As a result, after the first dequeue, the front pointer now will be moved ahead t0 the next location which is 1.

5.3 Array Implementation For Queue

Let us implement the queue data structure using C++.

```
#include <iostream>

#define MAX_SIZE 5

using namespace std;
```

```
class Queue {
private:
int myqueue[MAX_SIZE], front, rear;

public:
Queue(){
front = -1;
rear = -1;
   }

boolisFull(){
if(front == 0 && rear == MAX_SIZE - 1){
return true;
      }
return false;
   }

boolisEmpty(){
if(front == -1) return true;
else return false;
   }

void enQueue(int value){
if(isFull()){
cout << endl<< "Queue is full!!";
      } else {
if(front == -1) front = 0;
rear++;
myqueue[rear] = value;
cout << value << " ";
```

```
    }
  }
int deQueue(){
int value;
if(isEmpty()){
cout << "Queue is empty!!" << endl; return(-1); } else { value = myqueue[front]; if(front >= rear){      //only one element in queue
front = -1;
rear = -1;
        }
else {
front++;
        }
cout << endl << "Deleted => " << value << " from myqueue";
return(value);
    }
  }

  /* Function to display elements of Queue */
void displayQueue()
  {
int i;
if(isEmpty()) {
cout << endl << "Queue is Empty!!" << endl;
    }
else {
cout << endl << "Front = " << front;
cout << endl << "Queue elements : ";
for(i=front; i<=rear; i++)
cout << myqueue[i] << "\t";
cout << endl << "Rear = " << rear << endl;
```

```
    }
  }
};
int main()
{
   Queue myq;


myq.deQueue();      //deQueue


cout<<"Queue created:"<<endl; myq.enQueue(10); myq.enQueue(20);
myq.enQueue(30); myq.enQueue(40); myq.enQueue(50); //enqueue 60 => queue is full
myq.enQueue(60);


myq.displayQueue();


   //deQueue =>removes 10
myq.deQueue();


   //queue after dequeue
myq.displayQueue();


return 0;
}
```

Output:

```
Queue is empty!!
Queue created:
10   20   30   40   50
Queue is full!!
Front = 0
Queue elements : 10          20          30          40          50
Rear = 4

Deleted => 10 from myqueue
Front = 1
```

```
Queue elements: 20        30        40        50
Rear = 4
```

The above implementation shows the queue represented as an array. We specify the max_size for the array. We also define the enqueue and dequeue operations as well as the isFull and isEmpty operations.

5.4 Linked List Implementation for Queue:

Let us implement the queue in C++ using a linked list.

```
include <iostream>
using namespace std;
struct node {
int data;
struct node *next;
};
struct node* front = NULL;
struct node* rear = NULL;
struct node* temp;
void Insert(int val) {
if (rear == NULL) {
rear = new node;
rear->next = NULL;
rear->data = val;
front = rear;
     } else {
temp=new node;
rear->next = temp;
temp->data = val;
temp->next = NULL;
rear = temp;
   }
}
void Delete() {
```

```
temp = front;
if (front == NULL) {
cout<<"Queue is empty!!"<<endl; } else if (temp->next != NULL) {
temp = temp->next;
cout<<"Element deleted from queue is : "<<front->data<<endl;
free(front);
front = temp;
   } else {
cout<<"Element deleted from queue is : "<<front->data<<endl;
free(front);
front = NULL;
rear = NULL;
   }
}
void Display() {
temp = front;
if ((front == NULL) && (rear == NULL)) {
cout<<"Queue is empty"<<endl;
return;
   }
while (temp != NULL) {
cout<<temp->data<<" "; temp = temp->next;
   }
cout<<endl;
}
int main() {

cout<<"Queue Created:"<<endl;
Insert(10);
Insert(20);
Insert(30);
Insert(40);
Insert(50);
Display();
```

```
Delete();
cout<<"Queue after one deletion: "<<endl;
Display();


return 0;
}
```

Output:

```
Queue Created:
10      20      30      40      50
Element deleted from queue is: 10
Queue after one deletion:
20  30   40  50
```

5.5 Stack Vs. Queue

Stacks and queues are secondary data structures which can be used to store data. They can be programmed using the primary data structures like arrays and linked lists. Having discussed both the data structures in detail, it's time to discuss the main differences between these two data structures.

Stacks	Queues
Uses LIFO (Last in, First out) approach.	Uses FIFO (First in, First out) approach.
Items are added or deleted from only one end called "Top" of the stack.	Items are added from "Rear" end of the queue and are removed from the "front" of the queue.
The basic operations for the stack are "push" and "Pop".	The basic operations for a queue are "enqueue" and "dequeue".
We can do all operations on the stack by maintaining only one pointer to access the top of the stack.	In queues, we need to maintain two pointers, one to access the front of the queue and the second one to access the rear of the queue.
The stack is mostly used to solve recursive problems.	Queues are used to solve problems related to ordered processing.

5.6 Applications Of Queue

Let us discuss the various applications of the queue data structure below.

- The queue data structure is used in various CPU and disk scheduling. Here we have multiple tasks requiring CPU or disk at the same time. The CPU or disk time is scheduled for each task using a queue.

- The queue can also be used for print spooling wherein the number of print jobs is placed in a queue.
- Handling of interrupts in real-time systems is done by using a queue data structure. The interrupts are handled in the order they arrive.
- Breadth-first search in which the neighboring nodes of a tree are traversed before moving on to next level uses a queue for implementation.
- Call center phone systems use queues to hold the calls until they are answered by the service representatives.
- In general, we can say that the queue data structure is used whenever we require the resources or items to be serviced in the order they arrive i.e. First in, First Out.

Conclusion

The queue is a FIFO (First In, First Out) data structure that is mostly used in resources where scheduling is required. It has two pointers rear and front at two ends and these are used to insert an element and remove an element to/from the queue respectively.

Practice Problems:

Implement the queue that perform following functions:

1: Add a node (data member of type: string), Enqueue()

2: Delete a node, Dequeue()

3: Display the queue, Display()

In this activity modify above program by writing another function int LengthOfString(Queue *) “This function will return the number of characters in the string of nodes one by one and dequeue that node until queue is empty”.